

CASM: CONTEXT-AWARE SEMI-MARKOV POST-PROCESSOR FOR BEAT TRACKING

Zhanhong He¹, Hanyu Meng², Yaolong Ju^{3,*}

¹The University of Western Australia, Perth, Australia

²The University of New South Wales, Sydney, Australia

³Great Bay University, Dongguan, China

ABSTRACT

In music beat tracking, model-predicted activations must be decoded into a discrete, musically coherent event sequence. Direct peak picking closely follows local evidence but can retain spurious peaks or miss weak beats. Dynamic Bayesian networks (DBNs), a widely used structured post-processor, improve sequence consistency under predefined global tempo, meter, and transition constraints, but their behavior can depend strongly on these settings. We introduce CASM, a context-aware semi-Markov decoder that instead conditions its temporal constraint on local activation evidence. CASM also accounts for ambiguity among competing periodic interpretations, including half- and double-tempo alternatives. Deterministic safeguards prevent implausible outputs and preserve beat-downbeat consistency. Applied to fixed activations from three neural beat trackers (BeatThis, MSCNN, and TCN), CASM improves temporal continuity while preserving event-level F1 across the GTZAN and SMC datasets, without backbone retraining or dataset-specific retuning. Further analysis shows that CASM is less sensitive than the DBN baseline to the composition of the calibration data.¹

Index Terms— beat tracking, context-aware, semi-Markov decoder, post-processing, local periodicity

1. INTRODUCTION

Beats give music its temporal scaffold, so reliable beat tracking is crucial for music production, analysis, synchronization, and interactive systems [1]. More recently, beat tracking has become a key component of multimodal AI-generated content (AIGC), where the estimated beats serve as timing anchors for aligning generated content such as dance [2], video [3, 4], and music [5, 6]. Most music beat trackers operate in two stages: a network maps audio to frame-wise beat activations, and a post-processor converts these activations into discrete events. Despite recent attempts to remove the post-processor [7–9], many contemporary systems [10–14] still rely on it to obtain musically coherent estimates. No matter how accurate the neural activations become, direct peak picking makes decisions locally without considering the periodicity of beat sequences. Post-processing addresses this limitation by combining local activation evidence with the temporal regularity of music.

For post-processors, the key design question is how to balance temporal regularity against local activation evidence. A weak regularity constraint preserves the network’s predictions but may leave missed or spurious beats uncorrected. An overly strong constraint can instead suppress valid peaks under expressive timing or rapid

local tempo variation, which is common in classical music performances [15].

Many existing post-processors tilt this balance toward predefined temporal constraints. Early dynamic programming approach [16] assumes a single global tempo per track and penalizes inter-beat intervals (IBIs) that deviate from it. Conditional random fields (CRFs) [17, 18] learn sequence preferences that permit a range of local tempo variation while discouraging abrupt IBI changes. The dominant post-processor is the dynamic Bayesian network (DBN) [19, 20], which operates under pre-specified tempo ranges, meter states, and tempo transition probabilities. This design works well when these assumptions match the recording, but may require per-track or corpus-specific retuning when they do not [21].

The recently proposed PLPDP [22] shifts this balance to local evidence-conditioned temporal constraint. It derives a time-varying IBI and confidence from the predominant local pulse (PLP), which set the preferred period and how strongly it is enforced. However, because this confidence is derived from PLP peak heights, it reflects how strong the selected period is but not whether a competing period is nearly as plausible. A strong period may still be nearly tied with a metrical alternative, such as half or double tempo, and enforcing it strongly then risks committing to the wrong metrical level.

We introduce CASM, an offline context-aware semi-Markov post-processor. While semi-Markov decoding has shown leading performance and efficiency in real-time beat tracking [23], CASM operates offline on the complete activation sequence (with access to future context), so its performance is not directly comparable with that of causal online systems [24, 25]. The core philosophy is simple. Whereas PLPDP measures periodicity strength, CASM additionally accounts for periodicity ambiguity. It constructs a sparse path over activation-supported peaks and enforces strict temporal regularity when periodic cues are clear, but relaxes this constraint to let neural activations dominate when competing periods create ambiguity. A beat-count safeguard and a beat-synchronous meter decoder bound this behavior and keep the decoding deterministic. CASM offers three core advantages:

- **Backbone-agnostic and transferable.** CASM is a plug-in post-processor to cooperate with existing beat trackers and requires no backbone retraining. With a single fixed configuration, it improves every metric in our controlled comparison across BeatThis [8], MSCNN [26], and TCN [27] on both SMC [28] and GTZAN [29] datasets.
- **Low tuning burden.** Like PLPDP, CASM derives the preferred interval and constraint strength from each activation sequence, avoiding per-track or per-dataset BPM and meter retuning.
- **Open and reproducible.** We release CASM as an open-source Python package with the configuration used in our experiments and a *madmom*-like activation-to-event interface.

*Corresponding author, juyaolong@gbu.edu.cn.

¹Code and demo are available at <https://zhanh-he.github.io/casm-beat-tracking>

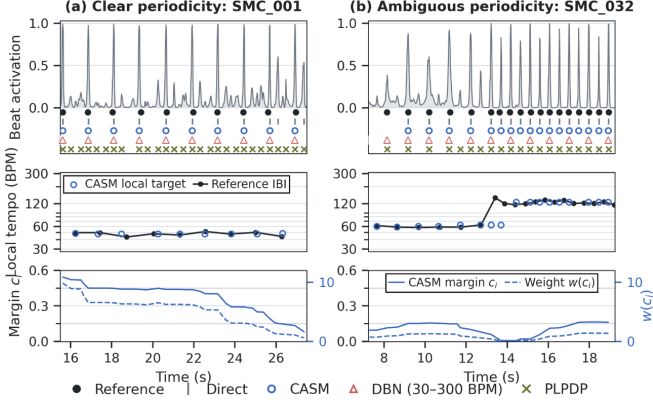


Fig. 1. CASM behavior on BeatThis activations. (a) CASM enforces stronger temporal regularization when periodicity is clear, but (b) closely follows direct peak-picking outputs when periodicity is ambiguous. $w(c_i)$ denotes the strength of temporal regularization.

2. METHODOLOGY

2.1. CASM overview

Figure 1 illustrates the adaptive role of CASM. When local periodicity is clear, CASM strengthens temporal regularity. When it is ambiguous, CASM relaxes the constraint and follows the neural activations more closely. CASM is an offline, model-agnostic post-processor operating on the beat and downbeat activations of a frozen neural tracker. It selects beats among activation peaks with a semi-Markov path that balances activation evidence and interval regularity (Sec. 2.2). At each candidate peak, CASM estimates the preferred local period and its margin over the strongest competitor (Sec. 2.3). The period sets the preferred inter-beat interval, while the margin controls how strongly it is enforced (Sec. 2.4). Decoding, safeguards, and downbeats complete the method (Sec. 2.5).

2.2. Semi-Markov beat path

CASM keeps every activation peak above a small threshold, including weak ones, as a candidate beat. Each candidate i receives an evidence score e_i , its beat logit scaled and clipped to a bounded range, so that $e_i > 0$ exactly when the activation exceeds 0.5. Two candidates can be consecutive beats only if the time Δ_{ij} between them lies within the interval support of 0.2–2 s (30–300 BPM). CASM selects the candidate sequence π that maximizes

$$\mathcal{S}(\pi) = \sum_{i \in \pi} e_i - \sum_{(i,j) \in \pi} D_{ij}, \quad (1)$$

where (i, j) runs over consecutive beats of π and D_{ij} is the duration cost of their interval. The two terms respectively reward strong beat evidence and penalize rhythmically inconsistent intervals.

This is the sense in which CASM is semi-Markov: each transition jumps from one beat directly to the next and scores the elapsed interval as a whole. A frame-level DBN tracks tempo and beat position as hidden states; CASM instead reads the preferred interval from the local context, so its decoding state is simply the last selected beat and no tempo states are needed. The dynamic programs of Ellis [16] and PLPDP [22] also score beat-to-beat intervals, but over every frame rather than only activation peaks. The scores are hand-specified, so CASM is a deterministic decoder rather than a generative hidden semi-Markov model or a learned semi-CRF.

2.3. Local period and period margin

To estimate the local period and its ambiguity, CASM compares the beat activation in an 8-s window centered on each candidate i with lagged copies of itself, using normalized autocorrelation over lags of 0.2–2 s. Because autocorrelation also peaks at multiples of the true period, a mild prior favors shorter lags. The best lag gives the local period τ_i with score q_i^* , and q_i^{alt} is the best score among lags more than two frames away from it. The period margin

$$c_i = \frac{q_i^* - q_i^{\text{alt}}}{q_i^*} \quad (2)$$

lies between 0 and 1. A large c_i indicates a clearly dominant period, whereas a small value indicates ambiguity from nearby or half- or double-time alternatives. It is a deterministic separation margin rather than a calibrated probability. Both τ_i and c_i are median-filtered over five consecutive candidates.

2.4. Margin-conditioned duration cost

Each interval (i, j) uses the context of both endpoints through geometric means, $\tau_{ij} = \sqrt{\tau_i \tau_j}$ and $c_{ij} = \sqrt{c_i c_j}$, so it is treated as unambiguous only when both of its ends are. Its duration cost is

$$D_{ij} = c_{ij} \frac{\lambda}{2\sigma(c_{ij})^2} \log^2 \frac{\Delta_{ij}}{\tau_{ij}}, \quad \sigma(c) = \sigma_0 + (1 - c)\sigma_u, \quad (3)$$

a Gaussian penalty on the log-interval that is zero at $\Delta_{ij} = \tau_{ij}$ and treats doubled and halved intervals alike. Here, $\lambda > 0$ controls the overall penalty strength, $\sigma_0 > 0$ sets the minimum timing tolerance, and $\sigma_u \geq 0$ controls relaxation under ambiguity. The margin thus plays two roles: the factor c_{ij} sets the penalty weight and $\sigma(c_{ij})$ its tolerance. As c_{ij} falls, the weight drops and the tolerance widens, so the cost vanishes at $c_{ij} = 0$ and stiffens rapidly toward $c_{ij} = 1$.

This single cost lets CASM add and remove beats relative to Direct: a weak candidate inside a doubled interval is selected if the two regular halves save more cost than its low evidence loses, and a strong off-grid peak is skipped if its irregular intervals cost more than its evidence gains. A large c_{ij} tilts these trade-offs toward a regular path, whereas a small one lets the activations decide, moving the path toward Direct. CASM can therefore recover weak but rhythmically supported beats without forcing an unreliable tempo grid.

2.5. Decoding, safeguards, and downbeats

A Viterbi-style dynamic program maximizes Eq. (1) and backtracks the best beat path, allowing a restart after an unfavorable prefix. Because the centered window and the backtracking use future activations, CASM is an offline decoder. As a count safeguard, CASM also forms the Direct path, i.e., the minimal peak picking of BeatThis [8] with seven-frame max pooling, a zero-logit threshold, and one-frame deduplication. If the CASM path has fewer than 0.85 or more than 1.8 times as many beats as Direct ($|\pi_{\text{CASM}}|/|\pi_{\text{direct}}| \notin [0.85, 1.8]$), indicating an implausibly sparse or dense path, the Direct path is returned. This rule is deterministic and requires no learned router.

Downbeats are decoded on the selected beat grid using a second dynamic program that allows bars of 2–7 beats and discourages unnecessary meter changes. If its output agrees poorly with the Direct downbeat peaks snapped to the same grid (F-measure below 0.6 at ± 70 ms), the snapped result is used, ensuring that every downbeat coincides with an output beat. All scalar parameters are calibrated once on the development folds (Sec. 3.3) and then frozen across tracks, datasets, and backbone models.

3. EXPERIMENTS

3.1. Evaluation protocol

Datasets and splits. We use the publicly released data from BeatThis [8], which contains 4,556 tracks from 18 beat-tracking datasets, including SMC, Hainsworth, Ballroom, Beatles, and others. This data collection provides preprocessed log-Mel spectrograms computed from 22.05 kHz mono audio at 50 fps, together with a fixed eight-fold partition to conduct cross-validation experiments. For each fold, the corresponding backbone checkpoint is trained on the remaining seven folds, ensuring that evaluated tracks remain unseen during training. Among these datasets, SMC provides beat but not downbeat annotations, and is widely regarded as the most challenging among them. Additionally, GTZAN provides 993 tracks as the held-out evaluation set, which is excluded from the cross-validation pool.

Backbones. We select three models whose implementations are publicly available and readily reproducible. For BeatThis, we use the original code and pretrained checkpoints [8]. MSCNN is adapted from the multi-scale, multi-task convolutional network of [26], which was designed for 60 s Bark-scale specific-loudness features. We modify it to accept the 30 s the unified log-Mel input and remove the task heads unrelated to beat and downbeat tracking. TCN was introduced by [27], and we use the re-implementation distributed with [26], which has been adapted from its original 5 s input to the same 30 s log-Mel input, with the tempo task head removed. All resulting backbones are trained on the same data and frozen before decoding performance comparison, ensuring that every post-processor receives exactly the same 50 Hz framewise beat/downbeat activations from a given backbone.

Evaluation. We conduct the same eight-fold cross-validation as in [8], where backbones are each evaluated on the held-out GTZAN and their corresponding out-of-fold SMC partitions. We use `mir_eval` [30] to compute F1 score with a ± 70 ms tolerance, CMLt, and AMLt. CMLt measures continuity at the correct metrical level, whereas AMLt additionally accepts standard off-beat and half- or double-level interpretations.

3.2. Post-processors and tempo support

Since our task is offline, all post-processors can use future context. We compare CASM with direct peak picking and three offline post-processors including DBN [19], PLPDP [22], and CRF beat detection [31]. Direct peak-picking has been defined in Sec. 2.5, and CASM uses a 30–300-BPM range by default. PLPDP and CRF are run with their released/default parameters. For methods without a joint beat/downbeat decoder, downbeats are decoded from the downbeat activation stream and snapped to the corresponding beat grid.

DBN receives separate treatment because its behavior is particularly sensitive to global tempo support. We use the standard joint madmom configuration [19]: meters {3, 4}, transition weight 100, observation weight 16, threshold 0.05, and the default 55–215-BPM range. Since many SMC excerpts have tempi below 55 BPM [21], Table 1 also explores a 30–300-BPM DBN variant in which only the tempo range is changed. For a matched diagnostic, we additionally report CASM with its BPM range restricted to 55–215, with all other settings unchanged (Table 2).

3.3. CASM calibration

CASM has no learned weights or random seed. Calibration is a simple grid search that keeps the configuration with the highest beat F1 on the calibration folds. The selected values depend on the calibration data used in each experiment, so we do not list them here. The stable released parameters and the full tuning history are available in our public GitHub repository. The configuration used in all main tables is calibrated once on SMC folds 1–7 (7F) and then frozen, so the same settings are used for every track, backbone, and dataset. SMC fold 0 is held out from decoder calibration and serves, together with GTZAN, as unseen evaluation data in Fig. 2.

To test sensitivity to calibration data, we calibrate on every subset of 1, 2, 4, or 7 folds from folds 1–7, denoted as 1F, 2F, 4F, and 7F, respectively. These scales yield 7, 21, 35, and 1 combinations, giving 64 calibration sets per decoder with no hand-picked combination. CASM and DBN are calibrated on each set using the same selection rule. For DBN, the grid covers the tempo range, the transition weight, which controls how strongly tempo changes are penalized, the observation weight, and the threshold. Every selected configuration is then evaluated on SMC fold 0, which tests transfer to unseen excerpts from the same collection, and on GTZAN using one fixed backbone seed, which tests transfer to a different dataset.

4. RESULTS

4.1. Comparison of post-processing methods

Table 1 isolates post-processor performance by holding the backbone activations fixed. CASM yields numerical improvements over the direct peak-picking baseline across every reported metric for all three backbones. While the 30–300-BPM DBN can yield large gains in continuity (particularly for GTZAN downbeats), it consistently degrades F1 accuracy. Furthermore, DBN performance is sensitive to tempo support: restricting its range to the default 55–215-BPM range improves some GTZAN metrics but causes large performance drops on SMC. Other alternatives such as PLPDP and CRF provide isolated benefits but fail to improve metrics consistently. Overall, CASM serves as a robust, plug-and-play upgrade that balances accuracy and continuity without requiring backbone-specific tuning.

Table 1. Post-processing comparison on fixed backbone activations. Backbone-only rows use direct peak picking. Post-processors use 30–300-BPM unless marked ^[55,215], the default DBN range which optimized for GTZAN-like music.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
BeatThis [8]	89.1	79.8	89.8	78.3	67.2	79.1	62.7	51.4	61.0
w. CASM	+0.3	+0.8	+1.2	+0.7	+3.6	+8.0	+0.3	+2.4	+3.6
w. DBN [19]	−0.5	−0.9	+0.2	−1.5	−0.6	+1.1	−1.6	+0.4	+3.5
w. DBN ^[55,215]	−1.0	+0.7	+1.3	−0.9	+6.2	+8.7	−5.2	−3.9	+1.5
w. PLPDP [22]	+0.1	+0.3	+0.3	+0.1	+0.2	+0.2	−0.9	−3.5	−0.3
w. CRF [31]	−0.6	−0.3	+1.4	−1.4	+1.9	+5.5	−1.2	+1.4	+1.2
MSCNN [26]	86.1	73.4	79.9	70.8	53.1	63.1	57.2	42.0	48.4
w. CASM	+0.4	+1.6	+2.4	+1.5	+12.5	+18.3	+0.3	+3.5	+6.8
w. DBN	−1.1	+0.2	+5.9	−0.9	+9.5	+14.6	−0.2	+2.2	+4.1
w. DBN ^[55,215]	−0.6	+4.0	+7.2	+1.8	+15.8	+21.2	−4.9	+0.3	+3.9
TCN [27]	87.1	73.8	81.7	62.4	24.6	59.9	52.9	30.1	36.0
w. CASM	+0.3	+1.3	+1.6	+2.1	+9.2	+13.4	+1.7	+4.5	+8.2
w. DBN	−1.9	−0.8	+4.4	−0.2	+8.9	+9.3	−0.4	+2.1	+5.9
w. DBN ^[55,215]	−2.6	−0.7	+5.0	+4.9	+23.2	+22.9	−0.9	−7.0	+2.3

4.2. Ablation of CASM components

Table 2 details the contribution of each CASM component by changing one design choice at a time. The adaptive margin is the most critical factor. With fixed stiffness, GTZAN gains turn negative and SMC gains shrink sharply. The margin should control both the penalty weight and the tolerance: using it for only one of them lowers F1 on both datasets and removes the GTZAN gains. A single-endpoint context keeps all gains positive but reduces them, especially on GTZAN. Without the count safeguard, F1 falls below the direct peak-picking while continuity improves, so the safeguard is what keeps CASM from trading F1 for continuity. Restricting the tempo range to 55–215 keeps every gain positive but makes them smaller. Unlike DBNs, CASM therefore needs less dataset-specific configuration, since a wider tempo range is better on both datasets.

Table 2. Ablation of CASM on BeatThis activations. Each row changes one design choice described in the subsection of Sec. 2. Values are changes from the direct peak-picking in percentage points.

Variant	Sec.	GTZAN (Beat)			SMC (Beat)		
		F1	CMLt	AMLt	F1	CMLt	AMLt
CASM (full)	–	+0.3	+0.8	+1.2	+0.3	+2.4	+3.6
55–215-BPM support	2.2	+0.1	+0.5	+0.6	+0.1	+0.4	+2.0
Fixed stiffness (no margin)	2.3	–0.3	–0.8	–0.8	+0.1	+0.7	+0.9
Margin on weight only	2.4	–0.1	–0.2	–0.2	–0.5	+0.9	+2.1
Margin on tolerance only	2.4	–0.3	–0.6	–0.6	–0.5	+0.9	+2.2
Single-endpoint context	2.4	+0.1	+0.3	+0.4	+0.2	+2.2	+2.6
No count safeguard	2.5	–0.1	+0.4	+0.5	–0.3	+2.5	+2.7

4.3. Comparison with state-of-the-art systems

Table 3 compares BeatThis with CASM against recent systems, all but one of which rely on DBN post-processing. The exception, BeatThis-MDM [32], uses a learned masked-diffusion decoder trained jointly with the BeatThis network. As it is data-driven, BeatThis-MDM improves on every GTZAN metric, whereas its SMC F1 and CMLt drop below those of BeatThis (direct). A learned decoder may favour their training-like data. CASM, in contrast, needs no retraining and works directly on the activations of any trained backbone, with the same parameters for every backbone. As Section 4.1 shows, CASM gives higher beat F1 than DBN and improves all SMC metrics across all tested backbones. DBN-based systems may therefore benefit from CASM. Those systems with large pre-trained frontends keep the best scores [11, 12], and replacing DBN with CASM in the latter is a natural next step.

Table 3. Comparison with recent beat-tracking systems. BeatThis (direct) and CASM results were obtained from our experiments, while all other results are reported from their original papers.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
HN-MERT [11]	89.7	81.4	94.3	77.4	71.6	87.2	61.7	–	–
HN-MusicFM	89.2	80.9	93.7	79.8	73.2	89.5	60.6	–	–
BeatFM-MERT [12]	<u>89.5</u>	<u>82.7</u>	93.9	76.7	70.2	85.4	61.3	–	–
BeatFM-MusicFM	89.1	80.6	93.5	79.6	74.4	<u>88.7</u>	60.5	–	–
BeatMamba [13]	88.9	82.3	94.3	–	–	–	58.7	46.8	62.4
BeatKAN [14]	88.2	78.1	92.3	–	–	–	59.8	48.4	64.0
<i>Above: with DBN post-processing below: without DBN post-processing.</i>									
BeatThis-MDM [32]	89.7	82.9	92.5	79.5	76.4	88.5	61.0	48.9	<u>64.3</u>
BeatThis [8]	89.1	79.8	89.8	78.3	67.2	79.1	<u>62.7</u>	51.4	61.0
BeatThis w. CASM	89.4	80.6	91.0	79.0	70.8	87.1	63.0	53.8	64.6

4.4. Sensitivity to calibration data

The sensitivity experiment keeps the evaluation sets fixed (the 0-fold of SMC, and the whole held-out GTZAN) and changes only the calibration data. For each calibration scale $k \in \{1, 2, 4, 7\}$, CASM and DBN are calibrated on every k -fold subset of SMC folds 1–7 (Sec. 3) and evaluated on the held-out SMC fold 0 and GTZAN. Thus, the spread at a given scale measures dependence on the chosen folds, while the shift of the distribution centre measures the effect of additional calibration data.

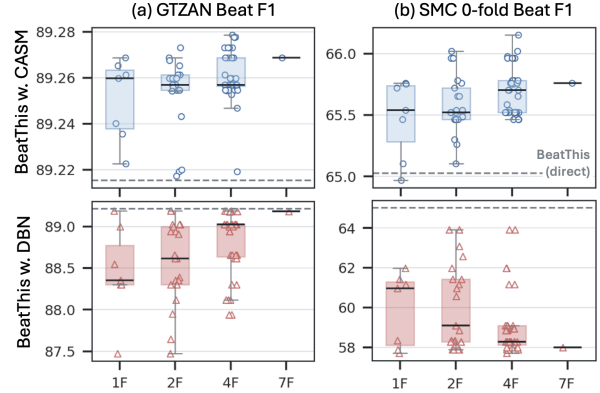


Fig. 2. Sensitivity to calibration data for CASM and DBN. Each point is one decoder configuration, calibrated on one of the 7/21/35 possible 1-/2-/4-fold subsets of SMC folds 1–7 (7F: all seven folds, a single configuration) and evaluated on SMC fold 0 and GTZAN. Because the subsets overlap, the boxes describe spread only. Dashed lines show the direct peak-picking performance as the reference.

Fig. 2 shows a clear difference in calibration stability. CASM remains tightly clustered across fold combinations and the median remains above Direct at every scale, even when calibrated on a single fold (1F). In contrast, DBN exhibits much wider variation, remains below Direct throughout, and its trends on the two sets diverge: more calibration folds improve GTZAN but degrade SMC. This is consistent with calibration-set over-specialisation in DBN. CASM therefore needs little calibration data, and the 7F setting used in the main tables reflects the full calibration pool rather than a favourable subset.

5. CONCLUSION AND FUTURE WORK

We proposed CASM, an offline, model-agnostic post-processor that balances immediate local beat evidence with broader rhythmic patterns. It can strengthen temporal regularity when the local evidence is clear and to rely more on activation evidence when it is ambiguous. We proved CASM works across different backbones using a single frozen calibration setup. Unlike most post-processors that force a trade-off, CASM uniformly enhances both local beat accuracy (F1) and global continuity (CMLt and AMLt). It is also less sensitive to calibration data than the DBN post-processor, indicating it much easier to tune. Future work can apply CASM to other state-of-the-art beat tracking systems as their code become open-source. Since CASM analyzes neural activations directly, it can ideally work with video, motion, and other multimodal beat tracking systems. Finally, we plan to integrate CASM into neural network training loops, similar to how PLPDP evolved into dPLP [33] or how SemiCRF is used in piano transcription [34].

6. ACKNOWLEDGMENTS.

The computational resources are supported by SongShan Lake HPC Center (SSL-HPC) at Great Bay University. This work was conducted while the first and second author were visiting student researchers at Great Bay University.

7. REFERENCES

- [1] H. Purwins, B. Li, T. Virtanen, J. Schlüter, S.-Y. Chang, and T. Sainath, “Deep learning for audio signal processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 2, pp. 206–219, 2019.
- [2] Z. Huang, X. Xu, C. Xu, H. Zhang, C. Zheng, J. Qin, and S. He, “Beat-It: Beat-synchronized multi-condition 3D dance generation,” in *Proc. Euro. Conf. Comput. Vision (ECCV)*, 2024, pp. 273–290.
- [3] J. Yu, Y. Wang, X. Chen, X. Sun, and Y. Qiao, “Long-term rhythmic video soundtracker,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 40339–40353.
- [4] X. Tong, Y. Zhu, J. Chen, C. Zhan, T. Wang, S. Zhang, N. Liu, T. Ge, D. Xu, X. Jin, F. Yu, and S.-C. Zhu, “Video echoed in music: Semantic, temporal, and rhythmic alignment for video-to-music generation,” in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026, vol. 40, pp. 25983–25991.
- [5] Y.-H. Lan, W.-Y. Hsiao, H.-C. Cheng, and Y.-H. Yang, “MusicConGen: Rhythm and chord control for transformer-based text-to-music generation,” in *Proc. ISMIR*, 2024.
- [6] K. Chen, Y. Wu, H. Liu, M. Nezhurina, T. Berg-Kirkpatrick, and S. Dubnov, “MusicLDM: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies,” in *Proc. ICASSP*, 2024, pp. 1206–1210.
- [7] T.-P. Chen and L. Su, “Toward postprocessing-free neural networks for joint beat and downbeat estimation,” in *Proc. ISMIR*, 2022, pp. 27–35.
- [8] F. Foscarin, J. Schlüter, and G. Widmer, “Beat this! accurate beat tracking without DBN postprocessing,” in *Proc. ISMIR*, 2024, pp. 962–969.
- [9] J. Bolt, J. Pauwels, and G. Fazekas, “Beat and downbeat detection: A reformulated approach,” in *Proc. ICASSP*, 2026.
- [10] J. Deng, Y. Ju, J. Yang, S. Lui, and X. Liu, “Efficient adapter tuning for joint singing voice beat and downbeat tracking with self-supervised learning features,” in *Proc. ISMIR*, 2024.
- [11] G. Ru, J. Wang, J. Zhao, Y. Wu, Y. Yu, N. Jiang, W. Wang, and W. Li, “HingeNet: A harmonic-aware fine-tuning approach for beat tracking,” in *Proc. ICME*, 2025, pp. 1–6.
- [12] G. Ru, J. Wang, J. Zhao, Y. Wu, Y. Yu, N. Jiang, W. Wang, and W. Li, “BeatFM: Improving beat tracking with pre-trained music foundation model,” in *Proc. ICME*, 2025, pp. 1–6.
- [13] G. Ru, Y. Yu, and W. Li, “BeatMamba: Bidirectional selective state-space modeling for efficient beat tracking,” in *Proc. ICASSP*, 2026.
- [14] Z. Zhang, G. Ru, and W. Li, “BeatKAN: An efficient and drum-attuned beat tracking method using kolmogorov-arnold networks,” in *Proc. ICASSP*, 2025, pp. 1–5.
- [15] H. Schreiber, F. Zalkow, and M. Müller, “Modeling and estimating local tempo: A case study on chopin’s mazurkas,” in *Proc. ISMIR*, 2020, pp. 773–779.
- [16] D. P. W. Ellis, “Beat tracking by dynamic programming,” *Journal of New Music Research*, vol. 36, no. 1, pp. 51–60, 2007.
- [17] T. Fillon, C. Joder, S. Durand, and S. Essid, “A conditional random field system for beat tracking,” in *Proc. ICASSP*, 2015.
- [18] M. Fuentes, B. McFee, H. C. Crayencour, S. Essid, and J. P. Bello, “A music structure informed downbeat tracking system using skip-chain conditional random fields and deep learning,” in *Proc. ICASSP*, 2019, pp. 481–485.
- [19] F. Krebs, S. Böck, and G. Widmer, “An efficient state-space model for joint tempo and meter tracking,” in *Proc. ISMIR*, 2015, pp. 72–78.
- [20] S. Böck, F. Krebs, and G. Widmer, “Joint beat and downbeat tracking with recurrent neural networks,” in *Proc. ISMIR*, 2016, pp. 255–261.
- [21] J. Ahn, T. G. Hwang, and M.-R. Jung, “The SMC blind spot: A failure mode analysis of state-of-the-art beat tracking,” 2026, <https://arxiv.org/abs/2605.12287>.
- [22] C.-Y. Chiu, M. Müller, M. E. P. Davies, A. W.-Y. Su, and Y.-H. Yang, “Local periodicity-based beat tracking for expressive classical piano music,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2824–2835, 2023.
- [23] M. Heydari, M. C. McCallum, A. F. Ehmann, and Z. Duan, “A novel 1D state space for efficient music rhythmic analysis,” in *Proc. ICASSP*, 2022, pp. 421–425.
- [24] P. Meier, C.-Y. Chiu, and M. Müller, “A real-time beat tracking system with zero latency and enhanced controllability,” *Trans. Int. Soc. Music Inf. Retr.*, vol. 7, no. 1, pp. 213–227, 2024.
- [25] M. Heydari and Z. Duan, “BeatNet+: Real-time rhythm analysis for diverse music audio,” *Trans. Int. Soc. Music Inf. Retr.*, vol. 7, no. 1, pp. 274–287, 2024.
- [26] Z. He, H. Meng, D. Huang, and R. Togneri, “Joint estimation of piano dynamics and metrical structure with a multi-task multi-scale network,” in *Proc. ICASSP*, 2026.
- [27] S. Böck and M. E. P. Davies, “Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation,” in *Proc. ISMIR*, 2020, pp. 574–582.
- [28] A. Holzapfel, M. E. P. Davies, J. R. Zapata, J. L. Oliveira, and F. Gouyon, “Selective sampling for beat tracking evaluation,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 20, no. 9, pp. 2539–2548, 2012.
- [29] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [30] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, “mir_eval: A transparent implementation of common MIR metrics,” in *Proc. ISMIR*, 2014.
- [31] F. Korzeniowski, S. Böck, and G. Widmer, “Probabilistic extraction of beat positions from a beat activation function,” in *Proc. ISMIR*, 2014, pp. 513–518.
- [32] F. Foscarin, F. Korzeniowski, and R. Vogl, “Masked diffusion enables coherent beat tracking,” in *Proc. ISMIR*, 2026.
- [33] C.-Y. Chiu, S. Strahl, and M. Müller, “dPLP: A differentiable version of predominant local pulse estimation,” in *Proc. ISMIR*, 2025, pp. 198–205.
- [34] Y. Yan and Z. Duan, “Scoring time intervals using non-hierarchical transformer for automatic piano transcription,” in *Proc. ISMIR*, 2024, pp. 973–980.